

SALESTORM: SCALABILITY & RELIABILITY DESIGN

Team Name : Binary Minds

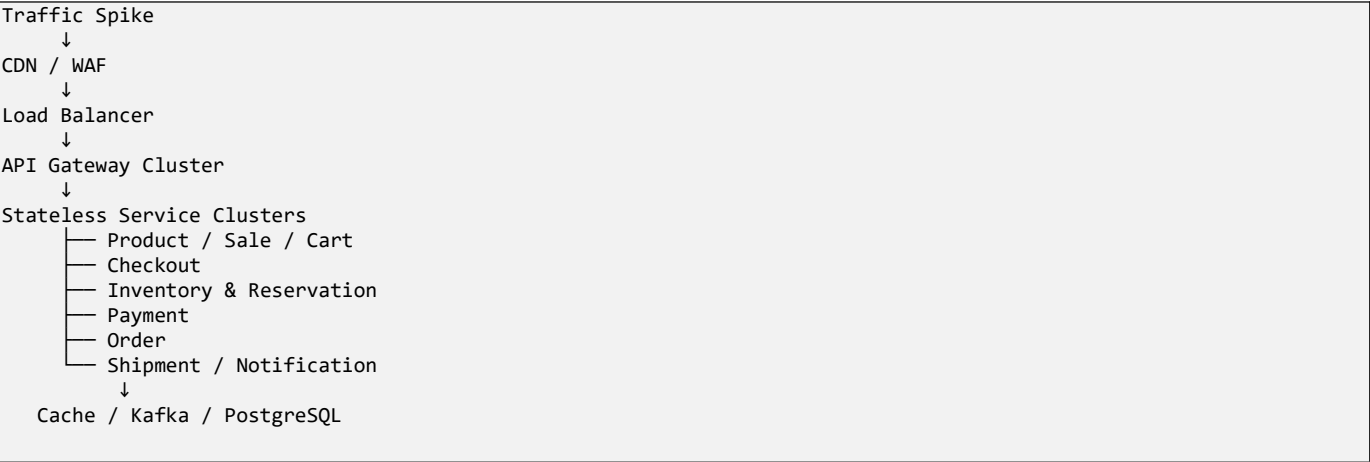
1. Purpose

This document defines how SALESTORM scales during flash-sale traffic and how it remains correct and recoverable when services, databases, messaging systems or external providers fail.

2. Traffic & Scale Requirements

Requirement	SALESTORM Target	Design Response
Normal traffic	~10,000 requests/sec	Horizontally scaled stateless services, caching and load balancing
Flash-sale peak	Reason about up to 500,000 requests/sec	CDN/WAF, load balancing, horizontal scaling, queueing and database scaling
Concurrency scenario	10,000 customers competing for 100 units	Controlled inventory contention and deterministic reservation
Traffic spike	Up to 50× normal traffic in stress reasoning	Rate limiting, queueing/waiting room and selective degradation

3. Core Principle



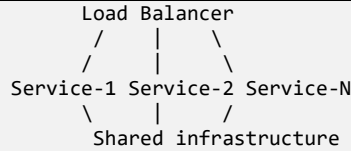
The architecture scales stateless components horizontally. Stateful correctness remains protected by transactional persistence and concurrency controls rather than by keeping business state inside an individual service instance.

SCALABILITY DESIGN

Handling the 10,000 → 500,000 req/s range

4. Horizontal Scaling

The primary scaling strategy is horizontal scaling of stateless services. Multiple instances sit behind a Load Balancer, allowing capacity to be increased without changing the service contract.

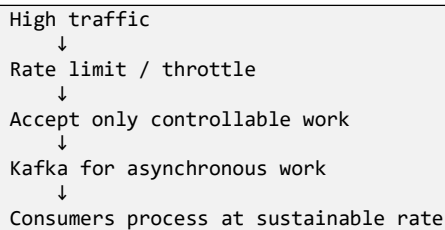


5. CDN & Caching

- Product and sale pages can be CDN-cached because browse traffic is read-heavy.
- Redis is used for fast access to frequently requested data.
- Caching reduces repeated database reads during the flash-sale spike.
- Critical inventory correctness is not delegated to cache alone; transactional inventory state remains authoritative in PostgreSQL.

6. Queueing & Backpressure

When downstream capacity is lower than incoming traffic, the system should avoid allowing the spike to propagate uncontrollably. Kafka provides asynchronous buffering for event-driven work, while the API edge can use rate limiting, throttling or a waiting-room approach for extreme checkout pressure.



7. Database Scaling

- PostgreSQL is the transactional source of truth for inventory, reservation, payment and order records.
- Read-heavy access can be reduced with Redis/CDN caching.
- Database scaling must preserve the inventory consistency boundary.
- Confirmed transactional records require durability and recovery support.

BOTTLENECKS & CONCURRENCY

Scale without sacrificing inventory correctness

8. Likely Bottlenecks

Bottleneck	Why it can bottleneck	Mitigation
Inventory reservation	Many users contend for the same limited stock	Concurrency-safe transaction, idempotency, fast sold-out path
Database	Large number of reservation/order/payment writes	Connection control, appropriate indexes, transactional design, scaling strategy
Payment gateway	External dependency with limited capacity or failures	Timeouts, retries, circuit breaker, idempotency and reconciliation
Order processing	Payment success can arrive faster than order creation	Kafka events, retries and eventual consistency
Notification provider	Large burst of outbound notifications	Asynchronous events and retry/DLQ handling

9. Inventory Hotspot

The flash-sale product creates a hotspot because thousands of customers compete for a small number of units. Scaling the Inventory Service alone does not solve this. The inventory update must remain concurrency-safe.

```
10,000 requests
↓
Inventory Service cluster
↓
Concurrency control
↓
PostgreSQL inventory transaction
↓
Only available units can be reserved

100 units → at most 100 successful reservations/sales
```

10. Sold-Out Fast Path

The requirements specify that out-of-stock rejections should be served in less than 100 ms without touching the database. The design therefore uses a fast availability signal/cache path for requests that can be confidently rejected, while authoritative reservation success still uses the transactional inventory mechanism.

11. Scaling Trade-off

More horizontal scaling increases capacity but does not automatically increase consistency. SALESTORM deliberately keeps the inventory consistency boundary strong even when service instances scale out.

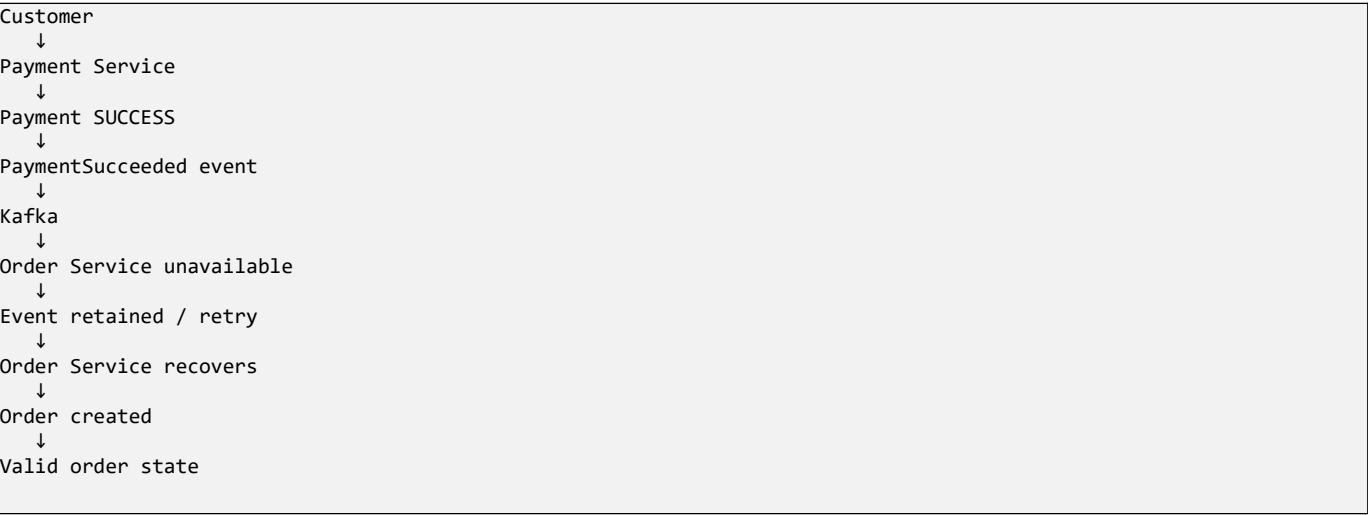
RELIABILITY & FAILURE HANDLING

Recovering without losing business correctness

12. Reliability Mechanisms

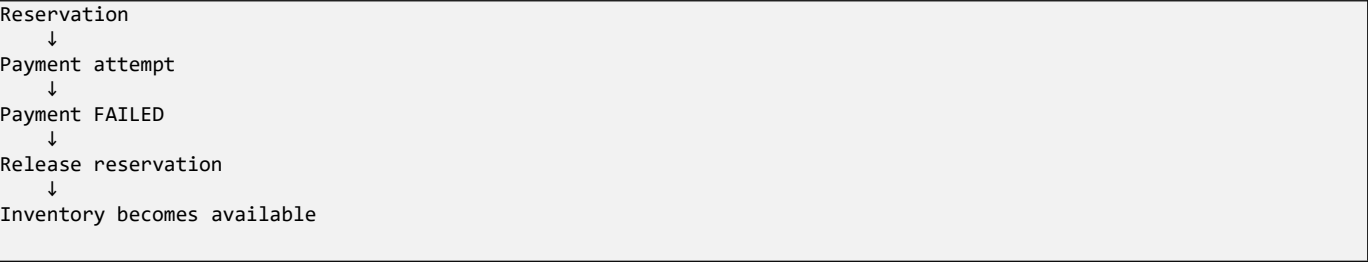
Mechanism	Purpose in SALESTORM	Example
Timeout	Prevent a slow dependency from blocking resources indefinitely	Payment gateway timeout
Retry	Recover from transient failures	Retry a failed event or safe payment status query
Circuit Breaker	Stop repeatedly calling an unhealthy dependency	Payment gateway outage
Idempotency	Make repeated requests/messages safe	Duplicate Buy Now or payment request
Dead Letter Queue	Preserve repeatedly failed async messages for investigation	Order event cannot be processed
Reconciliation	Repair state after distributed workflow uncertainty	Payment succeeded but order creation was interrupted
Compensation	Undo/release a prior business action when later work fails	Release reservation after payment failure

13. Payment Success → Order Service Failure



This supports the required eventual-consistency guarantee: every successful payment eventually reaches a valid order state. If the workflow cannot be completed automatically, reconciliation and compensation are used rather than silently losing the transaction.

14. Payment Failure



RELIABILITY TARGETS & RECOVERY

Requirements traceability

15. Key Non-Functional Targets

Requirement	Type
Successful reservations + sold units never exceed 100; available quantity never negative	Guarantee
Same idempotency key produces exactly one reservation, payment transaction and order	Guarantee
No double charge; every successful payment ends in an order or refund	Guarantee
Every successful payment eventually reaches a valid order state	Guarantee
Confirmed inventory, payment and order records survive node and DB-primary failure; RPO = 0	Guarantee
Reservation TTL 10 minutes; release within 30 seconds of expiry	Target
Reserve p95 < 300 ms and p99 < 800 ms at peak	Target
Sold-out response < 100 ms without database access	Target
Browse availability 99.95%	Target
Checkout availability 99.9%, with graceful queueing/waiting-room degradation	Target
Order within 60 s normally; within 5 min after downstream outage recovery	Target
DB failover RTO 5 min; payment gateway failure causes degraded mode, not data loss	Target
Horizontal scaling; design reasoned for 500,000 req/s peak and 10,000 req/s normal	Target
Authenticated purchase APIs, TLS 1.2+, rate limiting, validation, no raw card data, audit log, secrets vault	Guarantee
Metrics, structured logs, distributed traces and alerts for critical flows	Target

16. Guarantees vs Targets

Strict guarantees must hold even under failure: no overselling, no negative stock, no duplicate reservation/payment/order, no double charge, no paid customer without an order or refund, durable auditable business records, and authenticated/authorized purchase APIs.

Targets may degrade under extreme load: latency, availability, reservation-expiry timing, order-creation timing and notification speed.

RECOVERY SCENARIOS & TRADE-OFFS

How SALESTORM behaves under stress

17. Failure Scenario Matrix

Scenario	Expected Behaviour	Business Guarantee
Traffic surge	Scale stateless services; throttle/queue when necessary	System remains controlled; inventory correctness preserved
Inventory reaches zero	Fast out-of-stock rejection	Never oversell
Duplicate Buy request	Idempotency key returns the existing result	One reservation/order
Payment failure	Release reservation and update state	No stock permanently trapped
Payment timeout	Safe status check/retry/reconciliation	No duplicate charge
Payment gateway outage	Circuit breaker + degraded mode	No data loss
Order Service down 30 s	Publish event; retry after recovery	Successful payment eventually becomes order
Async consumer repeatedly fails	Retry, then DLQ	Message is not silently lost
DB primary failure	Fail over and recover durable records	RPO = 0 for confirmed inventory/payment/order records

18. Practical Test Case

The hackathon practical scenario uses stock = 100, 10,000 concurrent users, 95% payment success, 5% payment failure, 2% duplicate requests and Order Service unavailable for 30 seconds. The design must be able to trace success, failure, duplicate requests, temporary order-service failure and inventory exhaustion.

19. Architecture Trade-offs

Decision	Benefit	Trade-off / Cost
Synchronous reservation	Immediate inventory decision	Creates contention at the inventory boundary
Asynchronous order events	Resilient to temporary Order Service failure	Introduces eventual consistency and retry complexity
Caching	Reduces read/database load	Cache invalidation and stale-data concerns
Horizontal scaling	Handles higher traffic	More instances and operational complexity
Circuit breaker	Protects system from unhealthy dependencies	Requests may temporarily fail/degrade
Strong transactional inventory	Prevents overselling	Limits how freely inventory writes can be distributed

20. Final Design Statement

SALESTORM prioritizes correctness at the inventory and payment boundaries while using horizontal scaling, caching, queueing and asynchronous processing to absorb traffic. Reliability mechanisms—timeouts, retries, circuit breakers, idempotency, DLQ handling, reconciliation and compensation—ensure that temporary failures do not become permanent business inconsistencies.